

Roteiro de Prática Guiada — Momento 2 (50 min)

Aula 5 — Consumo de Web Services de Terceiros

Neste momento vocês executam e exploram um app mínimo já pronto, que consome dois serviços de terceiros encadeados: o projeto A05_03_Demo_Web_Services_Externos, fora do projeto do desafio. Rodem no próprio computador e acompanhem o código enquanto cada etapa é demonstrada. A implementação vem no Momento 3, no app da clínica.

Diferente das Aulas 3 e 4, hoje o servidor não é nosso e não roda na sua máquina. Isso muda tudo: não dá para reiniciar, não dá para corrigir, não dá para pedir um campo novo. Esta aula é sobre programar em cima de algo que você não controla.

Preparação do ambiente (5 min)

Descompactem A05_03_Demo_Web_Services_Externos.zip e, dentro da pasta do projeto:

```
npm install
npx expo start
```

- Hoje não há mock para subir: os dois serviços já estão no ar, na internet, mantidos por outras pessoas.
- Por isso mesmo, esta aula precisa de rede. Se a internet do laboratório cair, o app vai cair no tratamento de erro — o que também é conteúdo da aula.
- Nenhum dos dois serviços pede chave de API. É exceção, não regra: a maioria pede, e a Aula 12 (Mapas) vai exigir uma.
- Deixem o console do Metro visível. As mensagens técnicas vão para lá; as amigáveis, para a tela.

Parte A — ViaCEP: o caminho feliz e os outros (20 min)

Passo 1 — O serviço existe sem o app (5 min)

1. Abram no navegador: <https://viacep.com.br/ws/13560042/json/> e leiam o JSON cru.
2. Reparem no 'https' — não é opcional aqui, é o que o serviço oferece. Compare com a Aula 4, em que o mock local falava HTTP.
3. Reparem nos nomes dos campos: 'logradouro', 'localidade', 'uf'. Esse vocabulário é deles, não nosso — guardem isso para o Passo 3.

Passo 2 — As três respostas do ViaCEP (8 min)

4. No app, consultem o CEP 13560042. Endereço na tela: é o caminho feliz.
5. Agora consultem 99999999 — um CEP bem formado que não existe. A tela diz 'CEP não encontrado'.

6. Abram o mesmo endereço no navegador: `https://viacep.com.br/ws/99999999/json/`. O status é 200 OK, e o corpo é `{ "erro": "true" }`. Não é 404.
7. Este é o ponto mais importante da Parte A: `resposta.ok` era true, e mesmo assim não havia endereço. Sucesso de transporte não é sucesso de negócio.
8. Agora tentem 123 no navegador: `https://viacep.com.br/ws/123/json/`. Status 400 — e o corpo é uma página HTML, não JSON.
9. Por isso a função confere `resposta.ok` ANTES de chamar `.json()`. Na ordem inversa, o erro seria 'Unexpected token <', que não ajuda ninguém a entender o que houve.

Passo 3 — Traduzindo para o nosso vocabulário (7 min)

10. Acompanhem `consultarCep()` no `App.js`: ela devolve um objeto nosso, não o objeto do ViaCEP.
11. Reparem que 'localidade' vira 'cidade' na saída. Parece detalhe: é o que permite trocar de fornecedor mexendo em um arquivo só.
12. Reparem também na validação local dos 8 dígitos, antes de qualquer requisição. Ela economiza uma viagem à rede e uma unidade de cota de um serviço que não é nosso.

Parte B — Nominatim: quando os termos de uso mandam (18 min)

Passo 4 — Encadeando dois serviços (6 min)

13. Com o endereço na tela, toquem em 'Localizar no mapa'. Aparecem latitude e longitude.
14. Acompanhem o encadeamento: a saída do primeiro serviço é a entrada do segundo. Se o primeiro falha, o segundo nem é chamado — repare que o botão fica desabilitado.
15. Essas coordenadas são exatamente o que a Aula 12 (Mapas) vai precisar. O trabalho de hoje não é isolado.

Passo 5 — O 403 que não é erro de código (7 min)

16. Localizem a constante `USER_AGENT` no topo do `App.js` e o header que a usa dentro de `geocodificar()`.
17. Comentem essa linha do header e recarreguem o app. A resposta passa a ser 403 Forbidden.
18. Parem aqui. O código está correto, a URL está correta, a internet está funcionando — e mesmo assim não funciona. O que faltou foi cumprir os termos de uso do serviço.
19. A política do Nominatim exige que a aplicação se identifique. Ela também limita a 1 requisição por segundo e pede atribuição visível dos dados.
20. Descomentem a linha. E reparem no rodapé do app: o crédito ao OpenStreetMap está lá justamente porque a política pede.

Passo 6 — Sem resultado, com sucesso (5 min)

21. Consultem um CEP de uma rua com nome muito genérico e tentem localizar no mapa. Às vezes o Nominatim devolve 200 com uma lista vazia.
22. De novo o mesmo padrão do ViaCEP: 200, `resposta.ok` true, e nenhum resultado útil.

23. Acompanhem a checagem 'if (!resultados.length)'. Sem ela, o app tentaria ler resultados[0].lat e quebraria com 'undefined'.

Parte C — Quando o terceiro não responde (7 min)

24. Localizem TIMEOUT_MS no topo do App.js e a função buscarComTimeout().
25. Troquem o valor para 1 (um milissegundo) e recarreguem. Todas as chamadas passam a falhar com 'demorou demais para responder'.
26. Acompanhem o AbortController: ele é a forma padrão de cancelar um fetch. Sem ele, 'não responder nada' deixaria a tela em 'Carregando...' para sempre.
27. Devolvam o valor para 8000.
28. Agora desliguem o Wi-Fi da máquina e tentem consultar. A falha é outra — erro de rede, não tempo esgotado — e o app trata as duas.
29. Discutam a pergunta de projeto: se o serviço de CEP está fora do ar, o cadastro do paciente deve parar? A resposta certa é não — e é isso que o Momento 3 vai implementar.

Erros comuns — saibam reconhecer o sintoma

O que falta no código	O que vocês vão ver
a checagem de dados.erro (ViaCEP)	O cadastro é preenchido com campos vazios e ninguém percebe: o status foi 200 e o app assumiu sucesso.
conferir resposta.ok antes do .json()	'Unexpected token <' — o corpo do erro era HTML, não JSON. A mensagem não tem relação nenhuma com a causa real.
o header User-Agent (Nominatim)	403 Forbidden em toda chamada, com o código perfeito. O que falhou foi o termo de uso, não a lógica.
o timeout	A tela fica em 'Carregando...' para sempre quando o serviço não responde — o pior dos três modos de falha.
a checagem de lista vazia	'Cannot read property lat of undefined': o serviço respondeu 200 com uma lista sem itens.
espaçar chamadas em série	429 Too Many Requests, e depois um bloqueio temporário. Um Promise.all com 30 endereços viola a política de 1 req/s na hora.

Ao terminar

Confirmam que o demo consulta um CEP, localiza o endereço no mapa e mostra mensagem clara nos três modos de falha (erro do serviço, sem resultado, sem resposta). Quem não estiver com internet precisa resolver isso antes do Momento 3.